

Overview-and-Detail Visual Scrollbar for Long AI Chats

Visual design and implementation specification

Nathan Conklin

nathan.conklin@vt.edu

Department of Computer Science,
Virginia Tech

Abstract

Long AI conversations are difficult to navigate with a conventional scrollbar because the scrollbar communicates position but almost nothing about the content that occupies that position. This whitepaper specifies a visual conversation visualScrollbar that turns the right-side scroll control into an 80-pixel-wide overview of the complete chat session. The design uses an overview-and-detail relationship: the central conversation remains the detail view, while the visualScrollbar presents a compressed rendering of the same conversation. A proportional viewport indicator shows the portion currently visible, and faint full-width separators reveal prompt and response groupings as knowledge cards without converting the conversation into visually heavy boxed cards. Four interface concepts illustrate the intended behavior across code, architecture, analytics, and dashboard conversations. The paper concludes with the complete Codex implementation prompt reproduced verbatim so it can be executed directly.

1. Motivation and Design Goal

A conventional scrollbar answers two basic questions: where the user is in a document and how much of the document remains. That model works for homogeneous text, but it discards useful visual structure in an AI conversation. A long chat can contain prose, syntax-highlighted code, diagrams, charts, images, tables, and repeated prompt-response groups. Those differences create a recognizable visual rhythm in the main conversation, yet a normal scrollbar reduces the whole session to an undifferentiated track and thumb.

The proposed visualScrollbar preserves that rhythm at reduced scale. Its purpose is visual recognition and navigation; tiny text does not need to remain readable. A diagram in the detail view should have a corresponding miniature diagram in the overview. A large code block should remain identifiable by its shape and syntax colors. An image or chart should preserve enough of its dominant visual structure that the user can find it again.

The design fixes the overview width at exactly 80 pixels. The width does not vary with browser size, content type, session length, or zoom. The overview height follows the available height of the conversation panel, allowing the entire session to remain represented from top to bottom while the viewport indicator changes size as the ratio of visible content to total content changes.

2. Design Evolution

The concept developed through several iterations. The first design established the central idea: replace the thin scrollbar with a wider visual overview that shows miniature versions of the conversation. The next iteration tightened the overview-and-detail relationship by requiring the visualScrollbar to show the same content that appears in the chat; generic and decorative thumbnails were rejected. This synchronization is essential because the visualScrollbar is useful only when visual landmarks in the overview correspond to real landmarks in the conversation.

A later iteration introduced faint separators between prompt-response groups. These groups are called knowledge cards or thought cards in the source discussion. The separator communicates structure with little visual weight. It gives the eye a quiet cue that one prompt-response unit has ended and another has begun. The final visual refinement requires each separator to

span the full internal width of its panel while remaining clipped to that panel. Main-chat separators extend across the chat content panel. Visual separators extend across the visualScrollbar's internal width but never protrude beyond the 80-pixel boundary.

These iterations produce three design commitments that should remain stable during implementation:

- The overview is derived from the actual conversation, not from unrelated thumbnails.
- The viewport represents the visible-to-total content ratio and therefore changes height as the conversation grows or shrinks.
- Knowledge-card separators exist in both the detail view and the overview, with matched logical boundaries and intentionally low visual contrast.

3. Overview-and-Detail Interaction Model

The interface contains a detail view and an overview view that share one underlying conversation. The main chat pane is the detail view. It remains the primary place for reading, editing, selecting, copying, and interacting with content. The 80-pixel visualScrollbar is the overview. It compresses the entire conversation into the vertical space available beside the chat.

The two views must stay synchronized. The overview should preserve relative block size, dominant colors, major diagram shapes, image color patterns, code syntax colors, table density, whitespace, and the alternating rhythm of user prompts and assistant responses. Ordinary text can reduce to short line textures because legibility at 80 pixels is neither realistic nor required. Recognition is the goal.

The overview remains a session visualScrollbar. Topic labels, confidence scores, semantic branches, and other analytical layers are excluded unless those elements already exist in the conversation itself. The source design explicitly rejects those additions. The visualScrollbar is a visual compression and navigation surface for the real session.

4. Knowledge Cards and Structural Separators

A knowledge card is a logical prompt-response grouping. In the common case, one user prompt and the assistant material associated with that prompt form one card. A faint horizontal separator marks the boundary while preserving the feel of a continuous dark chat and avoiding large containers, background changes, or card outlines.

In the main conversation, each separator should be approximately one CSS pixel thick and only a few percent lighter than the surrounding dark background. It should span the full width of the chat content panel, from the panel's left internal boundary to its right internal boundary. It must remain inside that panel and should not stop at the width of an individual text column or message bubble.

The visualScrollbar repeats the same card boundaries. A separator appears at the vertically scaled position that corresponds to the real card boundary in the main conversation. It spans the full internal width of the visualScrollbar but is clipped to the visualScrollbar's bounds. This containment requirement is important because earlier concept images showed separators escaping the overview surface. The intended implementation uses the visualScrollbar's own coordinate system and explicit clipping so no separator enters the adjacent chat, native scrollbar area, browser chrome, or surrounding margin.

The faint separators provide a second layer of navigation information. The visual content shows what appears at a location, while card boundaries show how the conversation is segmented. Together they allow the user to scan a long session as a sequence of distinct conversational units.

5. Proportional Viewport Geometry

The viewport indicator is the visible rectangle inside the visualScrollbar. It communicates both the user's current location and the fraction of the complete conversation that is visible in the detail view. A fixed-size thumb would defeat this purpose, so viewport height is proportional to the visible chat height divided by the total chat content height.

The conceptual relationship is:

```
visibleRatio = visibleChatViewportHeight / totalChatContentHeight
visualScrollbarViewportHeight = visualScrollbarUsableHeight * visibleRatio
```

If the complete conversation fits on screen, the viewport should fill essentially the entire usable height of the visualScrollbar. If roughly half of the conversation is visible, the viewport should occupy roughly half of the visualScrollbar. For a very long conversation, the viewport becomes short. The implementation may clamp that visual height to a configurable minimum, initially around 20 to 24 pixels, so it remains easy to acquire and drag with a pointer.

A minimum visual height changes only the rendered control, not the underlying scroll mapping. Position calculations still need to map the top of the chat to the top of the visualScrollbar and the bottom of the chat to the bottom of the visualScrollbar. The implementation prompt therefore separates the physical viewport size from the scroll ratio and available viewport travel.

The conceptual scroll-position calculation is:

```
maximumChatScroll = totalChatContentHeight - visibleChatViewportHeight
scrollRatio = chatScrollTop / maximumChatScroll
maximumViewportTravel = visualScrollbarUsableHeight - visualScrollbarViewportHeight
viewportTop = scrollRatio * maximumViewportTravel
```

Non-scrollable and zero-height cases must be handled explicitly. The visual result should move smoothly without jitter, overshoot, or a visible gap at the bottom of the overview.

6. Two-Way Navigation

The visualScrollbar is an interactive navigation surface. Normal scrolling in the main conversation updates the visualScrollbar immediately. The user can also drag the viewport indicator to move through the detail view. Clicking a location elsewhere in the overview should navigate toward that corresponding point in the conversation, with centering used when practical.

The design leaves one interaction intentionally configurable: whether the mouse wheel should scroll the main conversation while the pointer is over the visualScrollbar. The implementation prompt tells Codex to inspect the existing application and ask before choosing behavior when the answer is ambiguous. That pattern is deliberate. The plan establishes functional intent but avoids overriding established application conventions without first examining the codebase.

7. Rendering and Synchronization

The most important rendering requirement is correspondence. The visualScrollbar should be based on the actual conversation model or DOM geometry. Independently generated preview content can break the mapping. A user who sees a colored architecture diagram in the detail view should be able to identify the same visual landmark at reduced scale in the overview.

Several rendering strategies can satisfy this requirement. A product could render reduced versions of existing presentation components, derive a dedicated visualScrollbar representation from the same conversation model, or use a canvas renderer fed by measured layout information. Direct DOM cloning can duplicate identifiers, event handlers, media loads, and

accessibility content, so it requires careful justification. The implementation strategy should follow the application's framework, virtualization approach, and existing ownership of the scroll container.

Geometry must remain current as content changes. New prompts, streaming responses, code blocks, images, diagrams, expanded content, regenerated messages, font changes, and browser resizing can all change total session height or card boundaries. The overview therefore needs a controlled invalidation strategy. Lightweight viewport movement can update continuously, while expensive reconstruction should be batched or throttled. Resize observers or framework-native lifecycle mechanisms can invalidate measurements when geometry changes.

8. Performance and Accessibility

A useful `visualScrollbar` cannot make ordinary chat scrolling feel slower. Continuously rasterizing or screenshotting the complete conversation on every scroll event would likely create unnecessary work. The design instead favors cached geometry, low-cost transforms for viewport movement, and reconstruction only when content geometry changes.

The implementation should also preserve existing accessibility paths. Native scrolling, keyboard scrolling, touch, and assistive technologies remain valid even if the `visualScrollbar` is unavailable. Mirrored miniature content should not cause a screen reader to announce the conversation a second time. The interactive viewport should receive an appropriate accessible description, and reduced-motion preferences should be respected. If the application already supports keyboard operation for custom scroll controls, the `visualScrollbar` should follow the same pattern.

9. Visual Design Examples

The four concept images below show how the same interaction model can work across very different chat content. They are examples of the intended relationship between the detail view, the 80-pixel overview, the proportional viewport, and the faint full-width separators. They should guide implementation behavior rather than serve as pixel-perfect specifications for unrelated parts of the host application.

9.1 Code Refactoring Conversation

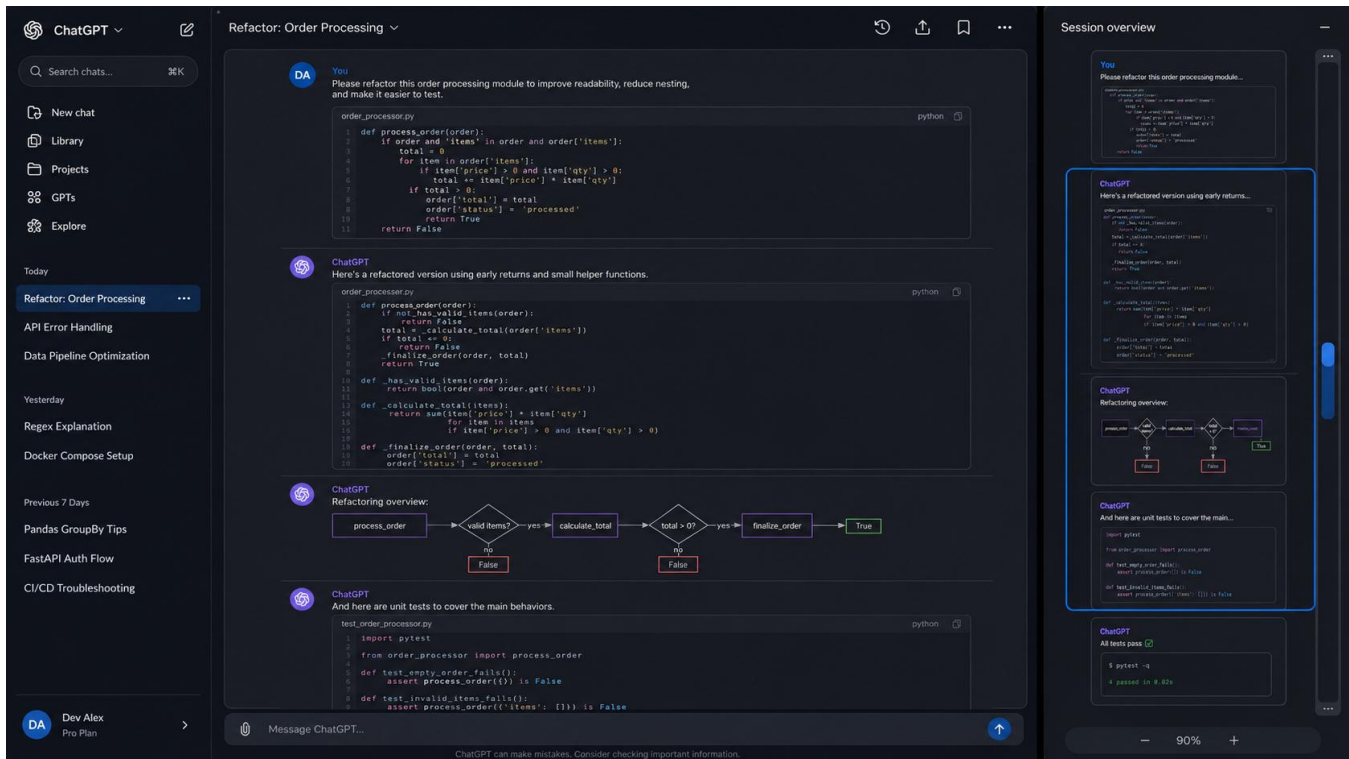


Figure 1: Code refactoring conversation with Python code, a flow diagram, tests, a synchronized right-side overview, and faint full-width knowledge-card separators.

The code-refactoring example demonstrates why visual correspondence matters. Large Python blocks create distinctive dense regions in the main conversation and in the overview. The miniature flow diagram and unit-test block form additional visual landmarks. The highlighted overview region indicates the part of the thread that is visible in the detail view.

9.2 Data Pipeline Architecture Conversation

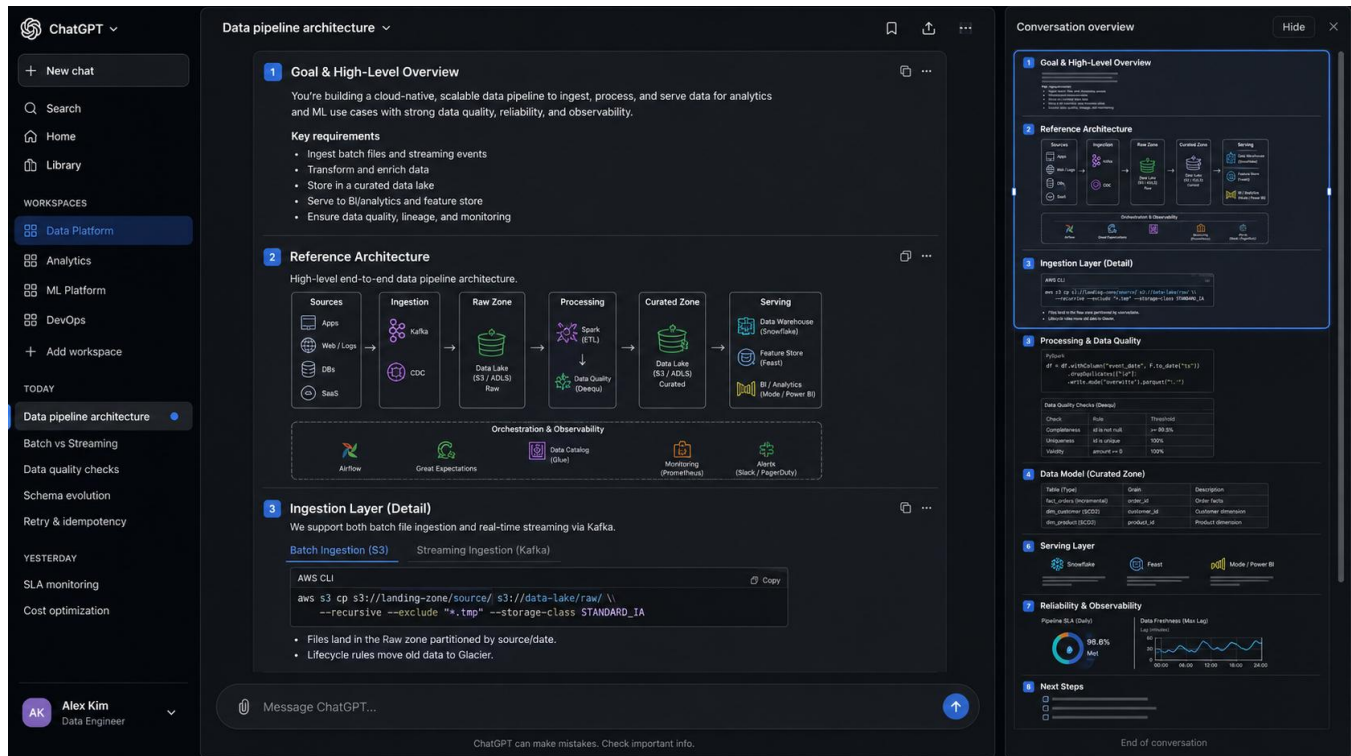


Figure 2: Data pipeline architecture conversation with an architecture diagram, code, tables, monitoring visuals, a synchronized overview, and faint knowledge-card separators.

The data-pipeline example combines diagrams, code, tables, and status graphics. The overview compresses those different block types while preserving their ordering and broad visual identity. Full-width separators establish card boundaries in both the central panel and the visualScrollbar.

9.3 Analytics Pipeline Developer Workspace

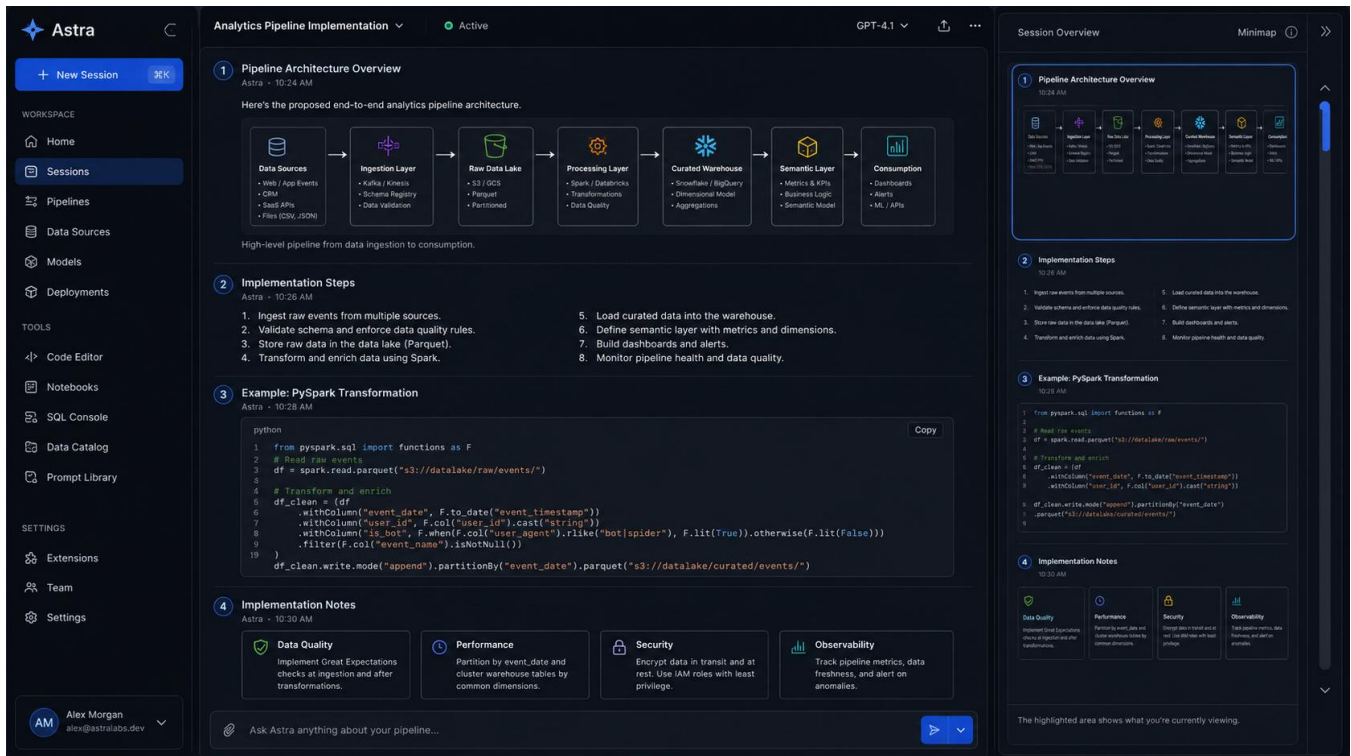


Figure 3: Dark analytics pipeline developer workspace with numbered implementation sections, architecture content, code, notes, and a synchronized conversation overview.

The analytics workspace illustrates how the concept can operate outside a literal ChatGPT clone. The same overview-and-detail behavior can sit beside an application-specific conversation or implementation workspace. The overview still represents the actual center-pane content and uses a bounded viewport tied to that content.

9.4 Dashboard Redesign Conversation

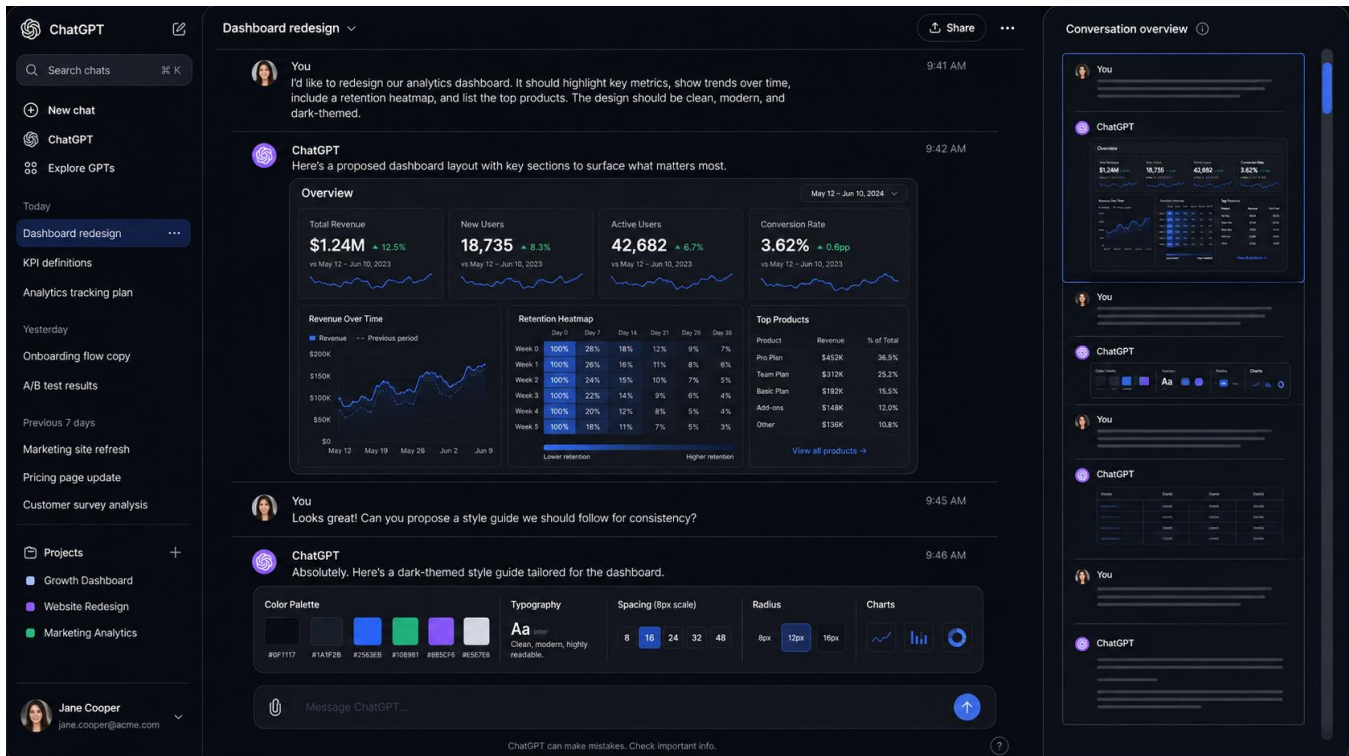


Figure 4: Dashboard redesign conversation with KPI cards, charts, a heatmap, tables, a style guide, and a synchronized right-side overview.

The dashboard example shows highly visual material. KPI cards, line charts, heatmaps, and style tokens remain recognizable even after aggressive compression. Preserving major colors and shapes prevents the overview from collapsing into generic gray lines.

10. Acceptance Criteria and Validation Strategy

The implementation prompt defines acceptance tests for short, medium, and very long conversations because viewport behavior depends directly on content length. A short conversation should produce a viewport that nearly fills the visualScrollbar. A conversation around twice the visible height should produce a viewport near half the visualScrollbar height. A very long conversation should produce a small but still usable viewport that reaches the exact top and bottom positions.

Mixed-content tests should include text, syntax-highlighted code, an image, a chart, a diagram, and a table. The overview should preserve the same sequence and major visual characteristics. Knowledge-card tests should confirm that separators span the full internal width of both panels but never escape their boundaries. Streaming and resize tests should confirm that the overview recalculates without visible jitter and that the visualScrollbar remains exactly 80 pixels wide.

The geometry should be isolated into testable functions where practical. Viewport height, viewport top, and coordinate conversions between chat space and overview space are deterministic calculations and can be tested independently from rendering. Integration or browser tests should cover drag navigation, click navigation, containment, resizing, and synchronization with the real scroll owner.

11. Implementation Process

The plan requires Codex to inspect the existing application before changing it. That inspection should identify the scroll-owning element, conversation data model, message grouping, rendering of code and media, virtualization, and responsive layout. Codex is explicitly instructed to ask as many questions as necessary instead of inventing interaction behavior or architectural boundaries.

After questions are resolved, Codex should present a short implementation design covering components, data flow, geometry, rendering, event handling, and performance. Major changes to existing scrolling should not proceed silently. The feature should then be implemented incrementally, tested, and summarized with the modified files, architecture, remaining assumptions, known limitations, and screenshots when the environment supports them.

12. Discussion and Open Implementation Choices

The design is intentionally specific about behavior and less prescriptive about internal rendering technology. That division allows the same feature to fit different front-end frameworks and conversation architectures. The fixed width, proportional viewport, synchronized overview, separator containment, and two-way mapping are product requirements. The exact rendering mechanism is an implementation choice that should follow evidence from the codebase.

The most likely engineering tension is fidelity versus cost. Rendering the complete conversation at miniature scale can become expensive for long sessions, especially with media-rich content or virtualized messages. Recognition is the objective; miniature readability is unnecessary. A specialized visualScrollbar renderer can preserve block proportions, colors, and shapes without reproducing every text glyph or interactive control.

A second open choice is how aggressively to preserve media detail. Static thumbnails can make images and diagrams easy to recognize, but the implementation should avoid duplicate network fetches and repeated decoding. Existing cached assets or low-resolution representations are preferable when available. Similar reasoning applies to code: preserving syntax-color distribution matters more than re-rendering every token at a scale where the text cannot be read.

13. Conclusion

The proposed visualScrollbar reframes chat navigation as an information-visualization problem. An 80-pixel overview of the actual conversation, a viewport whose height represents the visible fraction of the session, and faint knowledge-card boundaries give the user spatial context that a traditional scroll thumb cannot provide. The detail view remains the authoritative chat surface, while the overview provides recognition and navigation without adding unrelated semantic overlays.

The implementation plan below translates those design decisions into concrete requirements, geometry, tests, and process instructions for Codex. It is reproduced unchanged from the source conversation so it can be copied and executed directly.

Appendix A. Verbatim Codex Implementation Prompt

The following prompt is reproduced verbatim. Its wording, capitalization, spacing, formulas, and section structure are intentionally left unchanged so that it can be executed directly.

GOAL: Implement an Overview-and-Detail Chat Visual / Scrollbar

I want to replace or augment the traditional scrollbar for the main chat session with an 80-pixel-wide visual overview of the entire conversation.

The feature should use the information-visualization technique commonly called "overview + detail."

The main chat pane is the DETAIL view.

The new right-side scrollbar/visualScrollbar is the OVERVIEW view.

The overview must be a visual miniature of the actual chat session, not a generic set of markers. It should let the user understand the shape and structure of a long conversation at a glance and navigate through it naturally.

IMPORTANT IMPLEMENTATION INSTRUCTION

Before implementing this feature, inspect the existing application, DOM structure, component structure, scrolling behavior, and styling.

Ask me as many questions as you need in order to implement this correctly.

Do not make unnecessary assumptions about interaction behavior, component boundaries, existing scrolling architecture, or visual design when you can ask me instead.

If you discover ambiguities during implementation, stop and ask me additional questions.

I would rather answer several rounds of questions than have Codex quietly make design decisions that differ from what I intend.

=====

1. CORE USER EXPERIENCE

=====

The chat interface should have three conceptual regions:

1. Existing navigation/sidebar, if present.
2. Main chat conversation pane.
3. New visual overview scrollbar/visualScrollbar on the right.

The new overview scrollbar must have a FIXED WIDTH OF EXACTLY 80 PIXELS.

Its width must not increase or decrease based on:

- browser width,
- chat length,
- content type,
- zoom level,
- number of messages,
- viewport size.

The 80-pixel visualScrollbar should remain a stable visual/navigation element.

The vertical height of the visualScrollbar should correspond to the available height of the chat panel.

The visualScrollbar represents the ENTIRE chat session from beginning to end.

=====

2. OVERVIEW + DETAIL RELATIONSHIP

=====

The overview and the main chat must represent the same underlying content.

This is critical.

If the detail view currently contains:

- text,
- code,
- diagrams,
- charts,
- images,
- tables,

- formatted cards,
- user prompts,
- assistant responses,

then the corresponding location in the overview should visually resemble a tiny version of that SAME content.

Do not generate unrelated decorative thumbnails.

Do not substitute arbitrary charts, diagrams, code, colors, or blocks.

The overview should derive its visual representation from the actual chat session.

The user should be able to look at a colored diagram in the chat, look to the right at the corresponding portion of the visualScrollbar, and recognize the same diagram in miniature.

Likewise:

- a syntax-highlighted code block should produce a miniature area containing similar syntax colors,
- an image should appear as a tiny representation of that same image,
- a table should resemble that table,
- ordinary text should appear as dense miniature text lines or appropriate visual texture,
- a chart should retain its major colors and visual structure.

Think of this as a visual compression of the real conversation rather than a symbolic scrollbar.

=====

3. THE VIEWPORT INDICATOR

=====

Inside the 80-pixel visualScrollbar, display a viewport indicator.

The viewport indicator represents the portion of the complete conversation that is currently visible in the main chat pane.

The viewport indicator should be a subtle outlined or translucent rectangle over the visualScrollbar.

The content visible THROUGH or INSIDE the viewport indicator must correspond to the content visible in the main chat panel.

The viewport indicator must remain fully contained inside the 80-pixel overview panel.

Nothing belonging to the viewport indicator should extend outside the overview panel.

=====

4. VIEWPORT HEIGHT MUST BE PROPORTIONAL

=====

The height of the viewport indicator must dynamically represent the relationship between:

visible chat height
 versus
total chat content height

This means the viewport indicator MUST NOT have a fixed height.

For example:

If nearly the entire conversation fits on screen, the viewport indicator should occupy nearly the entire vertical height of the visualScrollbar.

If the user can currently see approximately half of the total conversation, the viewport should occupy approximately half of the visualScrollbar.

If the chat session is extremely long and the current screen contains only a small portion of it, the viewport should become correspondingly short.

Conceptually calculate:

```
visibleRatio =
    visibleChatViewportHeight / totalChatContentHeight
```

Then:

```
visualScrollbarViewportHeight =
    visualScrollbarUsableHeight * visibleRatio
```

Clamp the result so that the viewport never exceeds the visualScrollbar height.

If:

```
totalChatContentHeight <= visibleChatViewportHeight
```

then the entire session effectively fits on screen and the viewport should fill essentially the entire usable height of the visualScrollbar.

Allow only whatever tiny inset is necessary to render the viewport border cleanly.

=====

5. MINIMUM VIEWPORT SIZE

=====

Very long conversations may mathematically result in an extremely tiny viewport indicator.

Implement a configurable minimum viewport indicator height so the viewport remains usable with a mouse or pointer.

Do not hard-code this in several locations.

Use a named configuration value or CSS variable.

A reasonable initial implementation value would be approximately 20-24 pixels, but ASK ME if the existing product design suggests another value.

Even when the visual viewport must be clamped to a minimum size, scrolling calculations must still accurately represent the user's actual location within the complete conversation.

=====

6. VIEWPORT VERTICAL POSITION

=====

The viewport's vertical location in the visualScrollbar must correspond to the main chat's scroll position.

Conceptually:

```
maximumChatScroll =
    totalChatContentHeight - visibleChatViewportHeight
```

```
scrollRatio =
    chatScrollTop / maximumChatScroll
```

And:

```
maximumViewportTravel =
    visualScrollbarUsableHeight - visualScrollbarViewportHeight
```

```
viewportTop =
    scrollRatio * maximumViewportTravel
```

Handle all zero-height and non-scrollable cases safely.

When the chat is at the very top, the viewport must be at the top of the visualScrollbar.

When the chat is at the very bottom, the viewport must be at the bottom of the visualScrollbar.

Scrolling should produce smooth, stable movement without jitter.

=====

7. TWO-WAY NAVIGATION

=====

The visualScrollbar is not merely decorative.

It should be usable for navigation.

Support, unless the application's architecture makes one of these inappropriate:

A. DRAG VIEWPORT

The user can drag the viewport indicator vertically.

Dragging the viewport should scroll the main chat to the corresponding location.

B. CLICK MINIMAP

Clicking a location elsewhere in the overview should navigate the detail view toward the corresponding location in the conversation.

A good default behavior is to center the main viewport around the selected visualScrollbar location when possible.

C. NORMAL CHAT SCROLLING

Mouse wheel, trackpad, keyboard, touch, or normal scrolling in the main conversation should update the visualScrollbar viewport immediately.

D. OPTIONAL MINIMAP WHEEL

If appropriate for the current application, scrolling while the pointer is over the visualScrollbar may scroll the main conversation.

Ask me if there is ambiguity about this behavior.

=====

8. KNOWLEDGE / THOUGHT CARDS

=====

The conversation should be visually divided into conceptual groups that I am calling THOUGHT CARDS or KNOWLEDGE CARDS.

A card represents a logical prompt/response grouping in the conversation.

Typically this means a user prompt and the assistant material associated with that prompt belong to one knowledge card.

Do NOT turn these into visually heavy boxed cards.

The existing conversation should continue to feel like a continuous dark chat interface.

The card structure should primarily be communicated by an extremely subtle separator.

=====

9. MAIN CHAT SEPARATOR LINES

=====

Between adjacent knowledge cards, render a horizontal separator.

The separator should:

- be approximately 1 CSS pixel thick,
- be very faint,
- be low contrast,
- be dark,
- be only slightly distinguishable from the surrounding background,
- avoid drawing visual attention away from the conversation.

Think of something approximately 3-6% lighter than the surrounding dark background rather than a conventional visible divider.

CRITICAL:

The separator must span the FULL WIDTH OF THE CHAT CONTENT PANEL.

It should begin at the left internal boundary of the conversation panel and terminate at the right internal boundary.

It must remain inside the conversation panel.

Do not make the separator merely as wide as the text column or individual message bubble.

Do not allow it to extend into neighboring panels or underneath the visualScrollbar.

The result should subtly divide the entire conversation pane into knowledge-card regions.

```
=====
10. MINIMAP KNOWLEDGE-CARD SEPARATORS
=====
```

The 80-pixel overview must also display the same knowledge-card structure.

At the corresponding vertical boundaries between knowledge cards, render very faint horizontal separator lines inside the visualScrollbar.

These should visually match the separators in the main conversation, adapted to the reduced scale.

CRITICAL:

Each visualScrollbar separator must span the FULL INTERNAL WIDTH of the visualScrollbar.

However, it must NEVER extend outside the 80-pixel visualScrollbar.

Clip all visualScrollbar content to the visualScrollbar's bounds.

The visualScrollbar separators should not protrude into:

- the chat pane,
- a neighboring native scrollbar,
- browser chrome,
- margins outside the overview panel.

Prefer explicit clipping such as:

```
overflow: hidden
```

or an equivalent technique on the visualScrollbar rendering surface.

Separator coordinates should be calculated relative to the visualScrollbar's internal coordinate system.

The collection of faint separator lines should make it possible to look at the complete visualScrollbar and visually perceive that the conversation consists of many distinct knowledge cards.

```
=====
```


11. ALIGNMENT BETWEEN MAIN VIEW AND MINIMAP

=====

Knowledge-card boundaries in the main chat and visualScrollbar must represent the same logical boundaries.

For example, if the main session consists of:

```
Knowledge Card 1
-----
Knowledge Card 2
-----
Knowledge Card 3
```

then the overview should show those same three regions in compressed form.

The visualScrollbar separator should correspond proportionally to the location of the main conversation separator.

Do not independently guess separator positions in the overview.

Use the actual DOM/layout positions from the conversation whenever practical.

12. CONTENT SCALING IN THE MINIMAP

=====

Because the overview is only 80 pixels wide, text does not need to be readable.

The objective is visual recognition, not miniature reading.

Preserve:

- relative block sizes,
- major colors,
- diagram shapes,
- image color patterns,
- code syntax colors,
- table density,
- whitespace patterns,
- prompt/response rhythm.

Tiny ordinary text may be rendered as short line patterns.

Code should retain the visual distribution of syntax highlighting.

Images and diagrams should retain recognizable dominant shapes/colors whenever possible.

Do not enlarge the visualScrollbar in response to complex content.

The width always remains 80 pixels.

13. RENDERING STRATEGY

=====

First inspect the existing codebase and determine the most appropriate rendering strategy.

Possible strategies may include:

- miniature rendering derived from existing DOM nodes,
- mirrored/cloned presentation components rendered at a reduced scale,
- a dedicated visualScrollbar renderer built from the same conversation model,
- canvas-based rendering,
- another technique appropriate to the existing framework.

Do not blindly clone interactive DOM elements if that creates:

- duplicate IDs,
- duplicated event handlers,
- accessibility problems,
- expensive layout work,
- duplicated media loading,
- security problems,
- major performance problems.

The important requirement is visual correspondence between overview and detail, not a specific rendering implementation.

Ask me questions before choosing a substantially different representation.

```
=====
14. LIVE CONTENT AND STREAMING RESPONSES
=====
```

The visualScrollbar must remain correct while the chat session changes.

It should update when:

- a new user prompt is added,
- an assistant response streams in,
- a response grows vertically,
- a code block appears,
- an image finishes loading,
- a diagram renders,
- content expands or collapses,
- font/layout dimensions change,
- the browser is resized,
- the chat panel is resized,
- a message is edited or regenerated,
- cards are inserted or removed.

Use the framework's normal lifecycle mechanisms and/or appropriate observers such as ResizeObserver if needed.

Avoid excessive full-page recomputation on every token during streaming.

Batch or throttle expensive visualScrollbar layout work while preserving responsive movement.

```
=====
15. RESIZING
=====
```

The visualScrollbar width remains exactly 80 pixels.

Its height may change with the available application/chat-panel height.

Whenever its height changes:

- recompute overview scaling,
- recompute viewport height,
- recompute viewport position,
- recompute knowledge-card separator positions.

The viewport must remain correctly mapped to the current chat location after a resize.

```
=====
16. PERFORMANCE
=====
```

This feature must remain usable for very long conversations.

Avoid implementations that continuously screenshot or rasterize the complete page on every scroll event.

Scrolling the detail view should feel native and smooth.

Viewport-position updates should ideally happen using transform-based rendering or another low-cost mechanism.

Avoid causing repeated forced synchronous layouts.

Cache expensive measurements where appropriate.

Invalidate those measurements only when content geometry actually changes.

Throttle/debounce expensive overview reconstruction.

Do NOT noticeably slow down normal chat scrolling.

=====

17. VISUAL STYLE

=====

The target appearance is:

- very dark theme,
- understated,
- high-information-density,
- modern,
- polished,
- not visually noisy.

The visualScrollbar itself should blend into the UI.

The knowledge-card separators should be almost invisible until the user looks for them.

The viewport indicator can be somewhat more visible than the separators because it must communicate current location, but it should still fit the restrained dark theme.

Avoid bright divider lines.

Avoid large labels inside the visualScrollbar unless already part of the miniature representation.

Avoid turning the overview into a separate dashboard.

=====

18. VIEWPORT VS KNOWLEDGE-CARD LINES

=====

The viewport outline and card separators are different concepts.

Viewport:

- shows what portion of the conversation is currently visible,
- moves while scrolling,
- dynamically changes height,
- can be dragged.

Knowledge-card separators:

- represent conversation structure,
- remain tied to content,
- do not move independently,
- are extremely faint,
- appear in both overview and detail.

Make sure the viewport border does not obscure the faint card separators more than necessary.

=====

19. ACCESSIBILITY

=====

Do not make the visualScrollbar the only way to navigate.

Existing native scrolling, keyboard scrolling, and accessibility mechanisms must continue to work.

The visualScrollbar should not create duplicate screen-reader content for every miniature message.

Mark purely visual mirrored content appropriately so assistive technology does not read the complete conversation twice.

Provide an accessible description for the interactive viewport/navigation control.

Respect reduced-motion preferences.

Provide keyboard interaction for the viewport if the existing application accessibility architecture supports custom scroll controls.

=====

20. DOM AND LAYOUT REQUIREMENTS

=====

Do not modify the chat message content simply to make the visualScrollbar work.

Keep the visualScrollbar as a presentation/navigation layer over the existing conversation data and layout.

Create clear component boundaries.

For a component-based application, an architecture might resemble:

```
ChatConversation
KnowledgeCard
KnowledgeCardSeparator
ConversationOverview
OverviewContent
OverviewViewport
```

These names are illustrative rather than mandatory.

Use the conventions already present in the repository.

=====

21. CONFIGURABLE CONSTANTS

=====

Centralize important visual/behavioral values.

At minimum:

```
OVERVIEW_WIDTH = 80px
```

Potential configurable values:

```
MIN_VIEWPORT_HEIGHT
SEPARATOR_THICKNESS
SEPARATOR_OPACITY
VIEWPORT_BORDER_WIDTH
VIEWPORT_INSET
CLICK_NAVIGATION_BEHAVIOR
OVERVIEW_UPDATE_THROTTLE
```

Do not scatter magic numbers throughout the implementation.

=====

22. ACCEPTANCE TESTS

=====

The implementation is not complete until the following behaviors work.

TEST A – VERY SHORT CHAT

Create a conversation shorter than the visible chat panel.

Expected:

- overview width is exactly 80px,
- overview contains the entire short conversation,
- viewport fills essentially the entire visualScrollbar height,
- dragging cannot scroll beyond the available conversation.

TEST B – MEDIUM CHAT

Create a conversation roughly twice the visible panel height.

Expected:

- viewport height is roughly half the visualScrollbar height,
- moving the main chat halfway down places the overview viewport roughly halfway down.

TEST C – VERY LONG CHAT

Create a very long conversation.

Expected:

- viewport becomes small,
- it remains usable because of the minimum-height rule,
- it reaches the precise top and bottom positions,
- dragging accurately navigates the long conversation.

TEST D – MIXED CONTENT

Create a conversation containing:

- text,
- syntax-highlighted code,
- image,
- chart,
- diagram,
- table.

Expected:

- the overview visually resembles the same sequence,
- colors and block types line up with the main chat,
- the currently visible content corresponds to what appears under the overview viewport.

TEST E – KNOWLEDGE CARDS

Create at least five prompt/response knowledge cards.

Expected:

- faint full-width separators exist in the main chat,
- separators remain inside the main chat panel,
- corresponding faint separators appear throughout the overview,
- overview separators span its full internal width,
- overview separators NEVER extend outside the 80px visualScrollbar.

TEST F – STREAMING CONTENT

Stream a long assistant answer.

Expected:

- overview grows/updates as the answer grows,
- viewport size changes if total chat height increases,
- current scroll position remains stable,

- UI does not visibly jitter.

TEST G – WINDOW RESIZE

Resize the browser vertically and horizontally.

Expected:

- visualScrollbar remains exactly 80px wide,
- visualScrollbar height adapts,
- viewport size and position recompute correctly,
- knowledge-card boundaries remain aligned.

TEST H – BOTTOM OF CHAT

Scroll to the exact end of a long conversation.

Expected:

- the bottom of the viewport aligns with the bottom of the visualScrollbar,
- no overflow,
- no off-by-one gap large enough to be visually noticeable.

=====

23. TESTING

=====

Add automated tests appropriate to the existing codebase.

At minimum test the math separately from rendering.

Create pure functions where practical for:

```
calculateViewportHeight(...)
calculateViewportTop(...)
chatOffsetToOverviewOffset(...)
overviewOffsetToChatOffset(...)
```

Test:

- zero-height edge cases,
- short chats,
- exactly-one-screen chats,
- extremely long chats,
- minimum viewport clamping,
- top position,
- middle position,
- bottom position,
- resize behavior.

Add component/integration tests for:

- exact 80px width,
- separator containment,
- viewport containment,
- card boundary mapping,
- click navigation,
- drag navigation.

If the repository has Playwright, Cypress, or another browser test framework, add an end-to-end test for the complete scrolling interaction.

=====

24. IMPLEMENTATION PROCESS

=====

Please follow this sequence:

Phase 1:

Inspect the codebase and determine:

- what element actually owns scrolling,
- how chat messages are represented,
- how prompt/response groups can become knowledge cards,
- how code/images/diagrams are rendered,
- whether virtualization is already being used,
- how responsive layout currently works.

Phase 2:

Ask me all questions necessary to remove ambiguity.

Do not limit yourself to one question.

Ask as many questions as are necessary.

Phase 3:

Write a short implementation design explaining:

- components being changed,
- data flow,
- geometry calculations,
- visualScrollbar rendering strategy,
- event handling,
- performance strategy.

Show that design to me before making major architectural changes if the implementation would significantly modify existing scrolling behavior.

Phase 4:

Implement the feature incrementally.

Phase 5:

Add tests.

Phase 6:

Run the complete relevant test suite.

Phase 7:

Show me:

- what changed,
- files modified,
- architecture used,
- any assumptions that remain,
- known limitations,
- screenshots if the development environment supports them.

=====

25. DEFINITION OF DONE

=====

This goal is complete when:

The application has a fixed 80px-wide right-side conversation visualScrollbar.

The visualScrollbar is a recognizable compressed visualization of the actual complete chat session.

A dynamically sized viewport indicator shows the portion currently visible.

Viewport height is proportional to visible-content-height versus total-content-height.

Short conversations therefore produce very large viewport indicators.

Long conversations produce small viewport indicators.

The viewport moves proportionally with scrolling.

The overview can be used to navigate the conversation.

Main-chat content and overview content remain synchronized.

Prompt/response groups are represented as knowledge/thought cards.

Each knowledge card is separated by a very faint, full-panel-width line in the main conversation.

The same card boundaries are represented by very faint, full-internal-width lines inside the overview.

No visualScrollbar separator ever escapes the 80px overview boundary.

The feature remains performant during scrolling, resizing, and streaming responses.

Existing scrolling and accessibility behavior continues to work.